

Hands-on Activity No. <5.2>	
<Structures>	
Course Code: CPE007	Program: Computer Engineering
Course Title: Programming logic and design	Date Performed: 10/2/2025
Section: CPE11S1	Date Submitted: 10/4/2025
Name(s): Hanz Rhayven M. Pabalan	Instructor: Engr. Jimlord Quejado
6. Output	
SAMPLE #1 ANALYSIS we first include the basic structure along with #include <string> so we can use string variables. We then create a structure named Card that contains two strings: face and suit. Inside the main() function, we declare a struct variable Card a; and a pointer Card *aPtr;. After assigning values such as a.face = "Ace"; and a.suit = "Spades";, we set aPtr = &a; to let the pointer hold the address of a. When we use cout << a.face << " of " << a.suit;, the program prints "Ace of Spades." The pointer can also be used with the arrow operator, as in cout << aPtr->face << " of " << aPtr->suit;, which points directly to the members of the struct and produces the same result. Lastly, by dereferencing the pointer with (*aPtr).face and (*aPtr).suit, we can still access the members as if we were using the original struct variable. In summary, the dot (.) is used with the struct itself, the arrow (->) is used with a pointer, and the dereference operator (*) allows us to access struct members through a pointer. All three approaches will output the same result: "Ace of Spades." SAMPLE #2 ANALYSIS The program begins by building a structure with members that include both text and numerical data, such as an int for values like a number or ID and a string for the book's title. Two structural variables are declared to represent distinct books inside the main() function. The dot operator is used to assign values; for instance, book1.title gives direct access to the members of the structure. This procedure is then repeated for book 2, demonstrating how data can be effectively stored and arranged using numerous instances of the same structure. After then, the program completes its execution by returning 0. This illustrates how structures enable the grouping of similar data and how the dot operator is necessary to access and assign values to particular elements. SAMPLE #3 ANALYSIS A structure called Books with members for title, author, subject, and book_id is defined by the software. Two book variables are declared and given certain values in main(). After that, these values are shown by giving each structure to the printBook function, which uses the dot operator to print every detail. This demonstrates how related data may be stored in structures and how functions can access and display that data when the structure is supplied as an argument.	
7. Supplementary Activity	

```

#include <iostream>
using namespace std;

struct rectangle {
    double length;
    double width;
};

void compute (rectangle rect){
    double area = rect.length * rect. width;
    double perimeter = 2 * (rect.length + rect.width);
    cout << "Length: " << rect.length << endl;
    cout << "Width : " << rect.width << endl;
    cout << "Area  : " << area << endl;
    cout << "Perimeter: " << perimeter << endl;
}

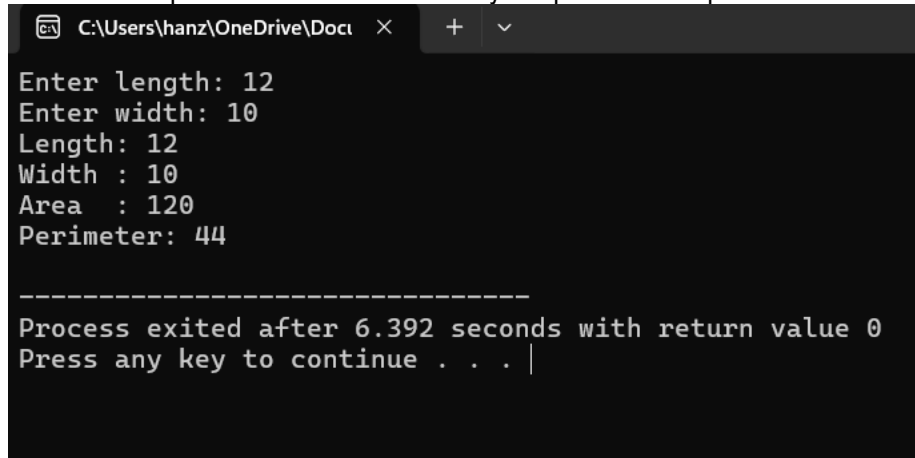
int main () {
    rectangle r;
    cout << "Enter length: ";
    cin >> r.length;
    cout << "Enter width: ";
    cin >> r.width;
    compute(r);

    return 0;
}

```

ANALYSIS

The Rectangle structure, which has two members length and width is defined by the application. The user enters the dimensions of a rectangle variable that is generated in the main() method. The function compute receives these variables and uses them to determine the area by multiplying length by width and the perimeter by adding length and width and multiplying by two. Lastly, the function shows the length, width, area, and perimeter of the rectangle. This demonstrates how functions can process related data to carry out practical computations and how structures can group related data.



```

C:\Users\hanz\OneDrive\Docu >
Enter length: 12
Enter width: 10
Length: 12
Width : 10
Area : 120
Perimeter: 44

-----
Process exited after 6.392 seconds with return value 0
Press any key to continue . . . |

```

```

struct1.cpp  multiple.cpp
1  #include <iostream>
2  using namespace std;
3
4
5  bool multiple(int num, int x) {
6      return (num % x == 0);
7  }
8
9  int main() {
10     int num, x;
11
12
13     cout << "Enter a number: ";
14     cin >> num;
15     cout << "Enter another number (x): ";
16     cin >> x;
17
18
19     if (multiple(num, x))
20         cout << num << " is a multiple of " << x << endl;
21     else
22         cout << num << " is not a multiple of " << x << endl;
23
24     return 0;
25

```

ANALYSIS

The program defines a function called `multiple` that uses the modulus operator to determine whether one integer is a multiple of another. Two integers are entered by the user inside `main()`, and the function receives these values. The function returns true, indicating that the first number is a multiple of the second, if the remainder of `num % x` equals zero; otherwise, it is not. `Cout` is then used to display the outcome. This illustrates how functions may be used to carry out straightforward logical checks and provide an understandable outcome.

```

C:\Users\hanz\OneDrive\Docu  X  +  v
Enter a number: 12
Enter another number (x): 3
12 is a multiple of 3

-----
Process exited after 64.29 seconds with return value 0
Press any key to continue . . . |

C:\Users\hanz\OneDrive\Docu  X  +  v
Enter a number: 13
Enter another number (x): 5
13 is not a multiple of 5

-----
Process exited after 8.484 seconds with return value 0
Press any key to continue . . . |

```

8. Conclusion

In my conclusion. I learned that using structures make it easier to group related data, like the book details, and we do this last week with the personal information, then I learned how pointers and operators like dot, arrow, and deference guve different ways to access the same information, and the functions are very useful because they let us separate task, like finding the area and perimeter of a rectangle or checking if a number is a multiple of another, overall these topics helped me understand how c++ can organize data and make programs more flexible and easier to manage.

9. Assessment Rubric